Contents

The Geometry of Arbitrage: Generalizing Multi-Hop DEX Paths via Möbius Transformations **1**
Abstract . . . 1
1. Introduction . . . 1
2. Constant Product Swaps as Möbius Transformations . . . 2
3. Path Composition and the Three-Scalar Recurrence . . . 3
4. Closed-Form Optimum and Profitability Criterion . . . 4
5. Verification . . . 4
6. Performance . . . 5
7. Discussion . . . 6
8. Conclusion . . . 6
Appendix: Equations Reference . . . 7

The Geometry of Arbitrage: Generalizing Multi-Hop DEX Paths via Möbius Transformations

Jeffrey Hartigan *April 2026*

Abstract

This paper generalizes the closed-form solution for optimal arbitrage between two constant product automated market makers (AMMs) to circular paths of arbitrary length. We show that every constant product swap is a Möbius transformation fixing the origin and that, because such transformations form a group under composition, an n-hop path collapses to a single Möbius transformation. The path is fully characterized by three scalar coefficients K, M, N that can be accumulated by a linear recurrence in $O(n)$. The optimal input is then given in closed form by $x_{\text{opt}} = (\sqrt{KM} - M)/N$, and the ratio K/M provides a zero-cost profitability test. A CPU implementation of the closed-form is 50–60× faster than a binary search baseline; promoted to GPU tensors, the recurrence evaluates one million candidate routes in approximately 50 ms. The principal contribution is the geometric framework itself, of which the two-hop formula is the simplest case.

---

1. Introduction

A previous derivation provided a closed-form solution for the optimal arbitrage input between two constant product liquidity pools:

$$x_{\text{ideal}} = \frac{k\sqrt{abcd} - ad}{k(bk + d)}$$

This result replaced the iterative binary search typically used in on-chain and off-chain arbitrage engines with a single algebraic expression. At the time, the formula was assumed to be a special case: multi-hop paths routed through three or more pools were expected to produce higher-degree polynomials without clean analytic solutions.

That assumption is incorrect. The algebra of constant product AMMs has additional structure that becomes visible under the right representation. Every constant product swap is a Möbius transformation, and the set of Möbius transformations forms a group under composition. A path of arbitrary length therefore collapses to a single Möbius transformation, parameterized by three scalars that admit a closed-form optimum.

The remainder of this paper formalizes this structure. Section 2 recovers the Möbius form from the standard constant product swap rule. Section 3 develops the matrix representation and the associated scalar recurrence. Section 4 derives the closed-form optimum and the profitability criterion. Section 5 verifies the formula against a binary search baseline on synthetic paths. Section 6 reports CPU and GPU performance. Section 7 discusses connections to related mathematical domains and practical implementation constraints.

---

2. Constant Product Swaps as Möbius Transformations

2.1 Transfer Function

A constant product swap with input reserve r, output reserve s, and fee factor $f \in (0, 1]$ maps input x to output y via:

$$y = \frac{sfx}{r + fx}$$

Writing the reserves in the notation of the original single-hop derivation, with a on the input side, b on the output side, and liquidity parameter k:

$$y = b - \frac{ab}{a + kx} = \frac{bkx}{a + kx}$$

Both expressions share the form $\alpha x/(\beta+\gamma x)$: a linear fractional transformation fixing the origin. This is precisely the shape of a Möbius transformation whose constant term vanishes.

2.2 Matrix Representation

A Möbius transformation $f(x) = (ax + b)/(cx + d)$ corresponds to the matrix:

$$\mathbf{M} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

acting on the extended real line under the standard projective action. For a constant product swap fixing the origin the upper-right entry vanishes, giving:

$$\mathbf{M}_i = \begin{pmatrix} s_i f_i & 0 \\ f_i & r_i \end{pmatrix}$$

The zero in the upper-right is preserved under multiplication of lower-triangular matrices of this form, so composition of any sequence of swaps preserves the fixed-point-at-origin property throughout the path.

---

3. Path Composition and the Three-Scalar Recurrence

3.1 Matrix Product

An n-hop path corresponds to the ordered product:

$$\mathbf{M}_{\text{total}} = \mathbf{M}_n \mathbf{M}_{n-1} \cdots \mathbf{M}_1 = \begin{pmatrix} K & 0 \\ N & M \end{pmatrix}$$

The upper-right entry remains identically zero at every stage of the multiplication. The total transfer function for the path is therefore:

$$l(x) = \frac{Kx}{M + Nx}$$

3.2 Scalar Recurrence

Maintaining the full 2×2 matrices is unnecessary and numerically noisier than operating on the three non-trivial entries directly. Let (K, M, N) denote these entries after accumulating the first i hops.

Initialization (first hop, reserves r_1, s_1, fee f_1):

$$K = s_1 f_1, \quad M = r_1, \quad N = f_1$$

Update for each subsequent hop i with reserves r_i, s_i and fee f_i:

$$K \leftarrow s_i f_i K, \quad M \leftarrow r_i M, \quad N \leftarrow r_i N + f_i K$$

The update for N uses the value of K prior to its update within the same step. These relations follow directly from the matrix product:

$$\begin{pmatrix} s_i f_i & 0 \\ f_i & r_i \end{pmatrix} \begin{pmatrix} K & 0 \\ N & M \end{pmatrix} = \begin{pmatrix} s_i f_i K & 0 \\ r_i N + f_i K & r_i M \end{pmatrix}$$

The recurrence is $O(n)$ — a single pass through the pool list with three multiplications and one addition per hop — which matches the theoretical lower bound given that every pool's reserves must be read at least once.

---

4. Closed-Form Optimum and Profitability Criterion

4.1 Optimal Input

Given $l(x) = Kx/(M + Nx)$, the profit for a circular path is $\pi(x) = l(x) - x$. Differentiating and setting the result to unity (the critical point of π):

$$\frac{dl}{dx} = \frac{KM}{(M + Nx)^2} = 1$$

Rearranging yields $(M + Nx)^2 = KM$, and taking the positive root gives:

$$\boxed{x_{\text{opt}} = \frac{\sqrt{KM} - M}{N}}$$

The formula is exact, requires no iteration, and scales as $O(n)$ in the number of hops.

4.2 Profitability Criterion

For $x_{\text{opt}} > 0$, the numerator $\sqrt{KM} - M$ must be positive, which requires $K > M$. Equivalently:

$$\frac{K}{M} > 1 \iff \text{profitable arbitrage exists}$$

The ratio K/M is the initial marginal exchange rate of the entire path — the rate on an infinitesimally small trade. If $K/M \leq 1$, no input size is profitable and no further optimization is warranted. This criterion is produced at no additional cost by the same recurrence that computes the optimal size, enabling a single-pass filter-and-size operation across large sets of candidate routes.

---

5. Verification

5.1 Recovery of the Two-Hop Formula

Applied to the two-pool case with Pool A $(r = a, s = b)$ and Pool B $(r = d, s = c)$, the recurrence yields:

$$K = bck^2, \quad M = ad, \quad N = k(bk + d)$$

Substituting into the general formula recovers the original single-hop result:

$$x_{\text{opt}} = \frac{\sqrt{bck^2 \cdot ad} - ad}{k(bk + d)} = \frac{k\sqrt{abcd} - ad}{k(bk + d)}$$

5.2 Synthetic Paths

The general formula was evaluated on synthetic circular arbitrage paths of varying length. For each path, the analytic result was checked against a direct simulation and a reference binary search optimizer.

Path	Closed-form x_{opt}	Profit	Binary search iterations
2-hop	7.921088	0.015955	54
3-hop	19.078671	1.118604	50
4-hop	85.827103	6.123027	50
5-hop	162.151803	13.873792	50

Table 1: Closed-form versus binary search on synthetic circular paths.

The closed-form and binary search agree to at least six decimal places on every path. The closed-form requires zero iterations, independent of path length.

---

6. Performance

6.1 CPU

Eliminating iteration yields a CPU speedup of approximately 50–60× over the binary search baseline across all tested path lengths. Binary search scales as $O(n \cdot I)$ with $I \approx 50$ iterations per path, while the closed-form scales strictly as $O(n)$.

6.2 GPU

The Möbius recurrence consists of arithmetic on three independent scalars per path, with no cross-path dependencies and no data-dependent branching. Promoting K, M, N from scalars to tensors of shape `(batch_size,)` yields a fully vectorized per-hop update across an arbitrary number of candidate routes.

Observed scaling behavior:

- Below approximately 10,000 parallel routes, GPU dispatch overhead dominates and CPU is faster.
- At 1,000,000 routes, the GPU implementation is more than 5× faster than the CPU implementation, evaluating the entire search space in roughly 50 ms.

In an end-to-end pipeline, a single batched computation produces the profitability check K/M, the optimal trade size, and the expected profit for every candidate path

simultaneously. Filtering, ranking, and execution operate on the resulting output tensor.

---

7. Discussion

7.1 Connections to Other Domains

Möbius transformations are the automorphisms of the Riemann sphere — the conformal maps that preserve circles and angles on the extended complex plane. The same group structure appears in several other domains:

- **Lorentz transformations**: The group $\mathrm{SL}(2, \mathbb{C})$ of Möbius transformations is isomorphic (up to sign) to the restricted Lorentz group, so the composition rule applied here is mathematically identical to the composition of relativistic velocities.
- **Continued fractions**: Möbius composition underlies the convergent structure of continued fractions, matching the simplification-under-chaining behavior observed when composing AMM swap functions.
- **Impedance networks**: Two-port network analysis in electrical engineering composes cascaded networks via the same 2×2 matrix formalism.

The constant product invariant $xy = k$ was chosen as a minimal viable AMM specification, with no reference to projective geometry. The Möbius structure is latent in the algebra and becomes visible only under the matrix representation.

7.2 Practical Considerations

- **AMM coverage**: The derivation assumes constant product AMMs. It does not apply unchanged to concentrated liquidity (e.g., Uniswap v3) or stableswap curves, which use different invariants.
- **Per-pool fees**: Paths may combine pools with heterogeneous fee structures. The recurrence handles this natively through the per-hop f_i.
- **State freshness**: The formula assumes static reserves throughout execution. Front-running by another trader on any pool along the path invalidates the computed optimum, reducing the problem to the standard MEV race condition.
- **Numerical overflow**: For long paths with large reserves the product KM can exceed double-precision floating-point range. Scaled arithmetic or logarithmic bookkeeping is required in that regime.

---

8. Conclusion

The closed-form solution for two-pool arbitrage is a special case of a general structure. Every constant product AMM swap is a Möbius transformation fixing the origin, and compositions of such transformations reduce to three scalar coefficients under a linear recurrence. The optimal input for a path of arbitrary length is given in closed form by $x_{\text{opt}} = (\sqrt{KM} - M)/N$, the profitability criterion is $K/M > 1$, and both

are produced in a single $O(n)$ pass over the path. The representation maps directly onto GPU tensor operations, enabling evaluation of one million candidate routes in roughly 50 ms.

The contribution is the geometric framework rather than any specific performance number, which will shift with hardware, path structure, and market conditions. Code, full derivation, and benchmarks are available at github.com/Alizerin-Jeff/analytic-arbitrage.

Appendix: Equations Reference

A.1 Constant Product Swap

$$y = \frac{sfx}{r + fx}$$

A.2 Matrix Representation of a Single Swap

$$\mathbf{M}_i = \begin{pmatrix} s_i f_i & 0 \\ f_i & r_i \end{pmatrix}$$

A.3 Path Recurrence

Initialization: $K = s_1 f_1,\ M = r_1,\ N = f_1$.

Per-hop update:

$$K \leftarrow s_i f_i K, \quad M \leftarrow r_i M, \quad N \leftarrow r_i N + f_i K$$

A.4 Total Transfer Function

$$l(x) = \frac{Kx}{M + Nx}$$

A.5 Optimality Condition

$$\frac{dl}{dx} = \frac{KM}{(M + Nx)^2} = 1$$

A.6 Optimal Input

$$x_{\text{opt}} = \frac{\sqrt{KM} - M}{N}$$

A.7 Profitability Criterion

$$\frac{K}{M} > 1$$